

MV-Scan: Relation-Aware Static Triage for Multi-Variable State Inconsistencies

VLADISLAV USATII, Rochester Institute of Technology, USA

YINXI LIU, Rochester Institute of Technology, USA

To circumvent the prohibitive gas costs of the Ethereum Virtual Machine (EVM), Decentralized Finance (DeFi) protocols are incentivized to adopt a dangerous architectural pattern: state redundancy. Instead of reading expensive global state (e.g., global time) in every transaction, protocols cache these values in local, redundant variables. We reveal that this optimization creates a systematic blind spot: Multi-Variable State-Inconsistency (MV-SI). In this defect class, a transition that partially advances state can violate that obligation even though every affected variable remains locally well-formed.

Although MV-SI sits inside the broader family of inconsistent state updates (ISU), existing automated analyses either target same-variable stale state or study multi-variable cases through manual taxonomies. They do not automatically infer latent cross-variable relations or test whether stale relations reach security- or economically relevant decisions; existing automated ISU checks still focus on whether individual state variables appear to miss updates, leaving MV-SI outside their detection scope.

We present MV-Scan, a static may-analysis for detecting candidate MV-SIs in Solidity programs. This paper systematizes the semantics of MV-SI, develops the first automated detection framework for its class, and provides the first large-scale empirical analysis of its prevalence in the DeFi ecosystem.

MV-Scan infers related persistent variables from predicate co-use and multi-return summaries, two recurring ways in which contracts reveal coherent protocol state. A storage-aware interprocedural analysis then tracks these relations across externally reachable execution contexts and identifies writers that update a proper subset of a relation. For each such writer, MV-Scan searches for an omitted member whose persistent value can subsequently influence control flow, another state update, or an external effect. The resulting witness connects the inferred relation, partial transition, stale-state consumer, sensitive operation, call path, and relevant storage-key constraints.

We evaluate MV-Scan on two complementary corpora. Among 116 published state-inconsistency findings, adjudication identifies 64 MV-SIs, 62 of which have accepted builds. MV-Scan strictly recovers 13 of these 62 cases (21.0%) when recovery requires agreement on the relationship, desynchronizing transition, and behavioral sink. On 65 buildable Web3Bugs projects, the default configuration achieves 20.5% candidate-level precision. A precision-oriented configuration reduces the structural candidate population from 2,868 to 550 and achieves 28.5% precision in an independently sampled audit. Analysis of 1,270 audited candidates shows that 97.6% contain a genuine protocol relationship, while 83.9% of non-bugs either lack a desynchronizing transition or restore consistency before the state is consumed.

Additional Key Words and Phrases: static analysis, smart contracts, state inconsistency, vulnerability detection

1 Introduction

Decentralized finance (DeFi) protocols store balances, debt, and other financial records in persistent variables, and protocol correctness usually depends on enforcing relationships between several variables. Calculating a fee, for example, requires both an amount to charge and an associated period. Accurate amounts produce incorrect fee calculations when paired with the wrong accounting period.

Kuiper, a protocol within the Ethereum ecosystem, directly illustrates this problem [13]. A contract calculates fees using its share supply, read through `totalSupply()`, and `lastFee`, the timestamp from which the next fee period is measured. After all shares are withdrawn, the supply becomes zero. If a user later deposits funds, the contract issues new shares but leaves `lastFee` unchanged. Subsequent fee calculations combine the new supply with the old timestamp,

Authors' Contact Information: Vladislav Usatii, Rochester Institute of Technology, Rochester, New York, USA, [TODO](#); Yinxi Liu, Rochester Institute of Technology, Rochester, New York, USA, [TODO](#).

Manuscript submitted to ACM

charging new holders for a preceding interval in which the basket had no shares. The resulting excess fee shares can severely dilute holdings. The obligation is specific: new holders should not be charged for that empty-supply interval. It does not require every mint to reset a legitimately accrued fee period.

We call this defect *multi-variable state inconsistency* (MV-SI). An operation changes only some related variables, commits the resulting inconsistency, and a later transaction relies on an omitted variable before reconciliation. In Kuiper, the relationship connects share supply to its fee period. The incomplete update changes the supply without advancing the timestamp and later fee calculations use stale values.

Not every delayed update is a defect. A protocol may synchronize related variables eagerly, as part of the operation that changes them, or lazily, when later interactions require them. Lazy synchronization is safe when the protocol completes that accounting before relying on the values. Synchronization does not always require equal values (e.g., a share count and a timestamp represent different quantities). Thus, the relationship validity downstream is significant.

MV-SI specializes the broader inconsistent-state-update problem studied by Li et al. [23]; consistency among related variables also has a long history in systems software [26]. Automatic invariant and property generators infer contract-specific relations from executions, code, templates, or natural language [1–6]. A different line of smart-contract research, including SAILFISH and DIVERTSCAN, studies schedule-dependent inconsistencies involving conflicting accesses to the same variable [7, 25]. Our focus is the specific report unit that connects an inferred relation, an omission-shaped writer, and a later consumer.

This setting introduces three key analysis challenges:

- (1) **Implicit relationships:** different functions or contracts may maintain relationships between variables without directly assigning one to another.
- (2) **Intentional partial updates:** a partial update may be deliberate, with a subsequent operation restoring consistency before the affected state is used.
- (3) **Ambiguous access relationships:** seemingly related accesses may refer to different users, mappings, or contracts.

A may-analysis must therefore identify (i) a plausible relationship, (ii) an update that may leave it violated, and (iii) a subsequent use that may be affected, while distinguishing these cases from benign or intentional updates.

We present MV-SCAN, a static may-analysis for uncovering MV-SI candidates. It infers variable relationships that jointly influence program conditions or one returned component. It searches for operations whose abstract matched writes cover a proper subset of the related group, applying a whole-root write-coverage heuristic. Finally, it follows reads of potentially outdated variables to modeled decisions and effects. The analysis follows resolved calls and tracks mapping keys, roots, senders, and abstract contract storage domains.

Each report connects a potentially incomplete update to code that could rely on an outdated variable. It identifies the inferred relationship, matched and omitted variables, downstream uses, and representative root-to-writer call-chain provenance. Since MV-SCAN assists auditors rather than inferring semantic intent, reports cannot prove whether inferred relationships are required, whether one execution realizes all recorded facts, or whether candidates are exploitable.

We evaluate MV-SCAN against published defects from two datasets, beginning with Web3Bugs [43]. Later manual review and adjudication classify 64 of the 116 inconsistent-state-update findings published by Li et al. [23] as MV-SI in our second dataset. Of these, 62 have accepted builds, and MV-SCAN strictly recovers 13, or 21.0% (Table 3) of them. Strict recovery requires identifying the relevant relationship, incomplete update, the omitted variable, and consequential downstream use. In Web3Bugs, we manually review 1,270 distinct structural buckets across the complete configuration

and four component ablations. Under the baseline configuration, 82 of 400 sampled buckets satisfy the MV-SI criteria, yielding 20.5% structural-bucket precision (Table 4).

The false positives identify the principal remaining challenge. Among the 1,270 distinct audited buckets across configurations, reviewers support the proposed relations in 1,239 (97.6%). This is a bucket-weighted conditional rate, not a census of distinct relation hypotheses. However, 522 nonbugs contain no persistent, desynchronizing partial update, and 291 restore consistency before the relevant use. These causes account for 813 of 969 nonbugs (83.9%; Table 7). Thus, recognizing related variables is an easier problem than determining whether incomplete updates persist until downstream usage.

The ablations expose a *precision-coverage tradeoff*. Disabling relationships inferred from branches reduces the candidate population from 2,868 to 550 and increases sampled precision from 20.5% to 28.5%. Our configurations use separately drawn, partially overlapping samples. The complete configuration has median analysis times of 7.61 seconds on successful Web3Bugs runs and 12.90 seconds on the historical-defect runs, although the largest tested projects require substantially more time and memory (Table 6). Based on these numbers, we position MV-SCAN as a static may-analysis to be used for triage in complex ISU scenarios involving several variables.

This paper makes four contributions:

- **We define the MV-SI analysis target.** We connect relationships among variables, persistent incomplete updates, and consequential uses, distinguishing actual defects from safe delayed synchronization and from the weaker evidence available to an automated detector.
- **An annotation-free candidate generator.** We combine relationship inference, partial-update analysis, and tracking of later uses while accounting for resolved calls, mapping keys, roots, senders, and abstract storage domains.
- **Reports that support manual validation.** We organize candidates around incomplete updates and provide their affected variables, later uses, call paths, and explicit assumptions, while distinguishing detector reports from the groups used in evaluation.
- **An empirical evaluation and diagnosis.** We measure strict recovery of published defects, structural-bucket precision, four component ablations, scalability, and reproducibility. Our error analysis identifies persistent partial updates and intervening synchronization as the principal remaining limitations.

2 Background and Motivating Example

2.1 Persistent state and redundant protocol views

Ethereum transactions are atomic, but contracts preserve state across transactions in persistent storage and can read other contracts. Storage and recomputation costs encourage incremental accounting, checkpoints, and derived views [18]. Under EIP-2929, a cold storage read is charged 2,100 gas. Under EIP-2200, setting an originally clean zero slot to nonzero costs 20,000 gas, with the cold-access surcharge applied when relevant [10, 34]. By contrast, `block.timestamp` is an environmental opcode value, not an expensive persistent-storage read.

Protocols use indexes, snapshots, checkpoints, caches, and local mirrors. For example, a global interest index may be updated once while each account stores the index when it was last settled. In another instance, governance power may be cached. A protocol may even record remote token balances locally so downstream operations can use an accounting view without repeating a cross-contract read. Such designs are not inherently incorrect. They define *synchronization boundaries*: points at which two or more representations must satisfy the relation assumed by downstream logic.

```

157 mapping(address => uint256) balance;
158 mapping(address => uint256) votingPower;
159
160 function canVote(address u) external view returns (bool) {
161     return balance[u] > 0 && votingPower[u] >= MIN_POWER;
162 }
163
164 function withdrawAll() external {
165     address u = msg.sender;
166     uint256 amount = balance[u];
167     balance[u] = 0; // votingPower[u] omitted
168     token.transfer(u, amount);
169 }
170
171 function vote(bytes32 proposal) external {
172     address u = msg.sender;
173     require(votingPower[u] >= MIN_POWER);
174     hasVoted[proposal][u] = true;
175 }
176

```

Fig. 1. Motivating case study. The withdrawal updates one member of a per-account relation, while a later control decision consumes the omitted voting weight.

The relevant relation is not always equality. If shares and assets are connected by an exchange rate, consistency may require a proportional equation. A risk module may require collateral value to exceed debt by a threshold. A global timestamp and per-market checkpoint may be related by an ordering constraint. Accordingly, this paper uses *relation* for the protocol condition that links distinct persistent locations, and *reconciliation* for a transition that restores the condition required before a later use.

2.2 Motivating example

Figure 1 is a semantic illustration of a minimal governance cache. It is not an end-to-end walkthrough of frozen-detector output: the shown `canVote` view is not a state-changing root, and the snippet contains no retained root that calls it. The protocol keeps a token balance and a separately stored voting weight for each account. The intended relation is that `votingPower[u]` reflects the voting entitlement derived from `balance[u]`, while `vote` consumes the cached weight directly.

The code contains three distinct questions. First, are `balance[u]` and `votingPower[u]` related? Their joint influence on the value returned by `canVote` demonstrates potential, but the final semantic judgment might vary. Second, can `withdrawAll` commit a state in which only `balance[u]` is updated? The assignment is locally partial, but the answer would change if a modifier, internal call, or guaranteed post-dominator always synchronized `votingPower[u]` before the transaction completed. Third, can the stale member affect behavior before reconciliation? Here, `votingPower[u]` reaches the predicate that authorizes a persistent governance write.

If all three answers are positive, the sequence `withdrawAll()` followed by `vote(p)` is an MV-SI: the writer leaves the relation partially advanced, and the reader acts on the omitted state. If `withdrawAll` invokes a synchronization routine on every returning path, there is no persistent partial transition. If `votingPower` is retained but never influences a decision or effect, there is no behavioral consumption. If governance intentionally grants lifetime voting rights

independent of current token holdings, the apparent relation is unsupported. The same syntax can therefore represent a defect, a benign cache, or two unrelated variables.

2.3 Why existing detection anchors are insufficient

Existing analyses typically begin by detecting conflicting accesses to one state location and then asking whether ordering, reentrancy, or flow divergence makes the conflict exploitable [7, 25]. In our case study, however, the two relevant accesses are to different mapping slots. A unary missed-update rule may flag votingPower, but it cannot by itself identify what balance to track or which transition breaks that relation. General static analyzers and symbolic executors can reason about these operations once given properties and targets [14, 15, 29]. Standard analyses miss the multi-variable relation and the semantic boundary where divergence matters.

To be useful, MV-SI analysis should preserve keyed identities such as balance[u] and carry argument and sender bindings across calls. The frozen implementation distinguishes abstract contract storage domains, but not concrete deployed instances of the same declaration. We must also distinguish store instructions from effective value changes and benign reads from reads whose value may influence behavior. These complexities motivate the analysis objective.

3 Multi-Variable State Inconsistencies

3.1 Failure pattern and temporal scope

We study an omission-shaped failure across transaction boundaries. Several persistent variables participate in one protocol obligation; one transaction changes a nonempty proper subset, commits before restoring that obligation, and a later transaction consumes an omitted variable while the relation remains invalid. A temporary divergence that is always reconciled before the writer returns is outside this target. So is a same-transaction use that occurs before eventual reconciliation. Such intermediate-state effects can be real because calls within a transaction observe intermediate state, but the frozen oracle and detector’s whole-root gate were not designed to label them consistently. We therefore do not reinterpret the existing labels as validating that broader target.

This boundary also makes the omission focus explicit. A bug that updates every related variable to mutually incompatible values may be relational, but it is not an MV-SI as defined here.

Let a concrete execution start from reachable protocol state S , execute writer transaction τ_w , and commit state S' . For related concrete persistent cells M , define the semantic change set

$$\Delta_M(S, S') = \{\ell \in M \mid S(\ell) \neq S'(\ell)\}.$$

This is different from executing a store instruction: $x = x$ stores without changing a value, and a transaction can change a value and restore it before commit.

An MV-SI exists in one feasible execution from a reachable protocol state when all four conditions hold:

- **C1 (distinct persistent variables).** M contains at least two distinct mutable persistent cells whose values coexist across transaction boundaries.
- **C2 (supported relation).** Evidence beyond the co-influence signal that generated the hypothesis supports a protocol obligation Φ over M . Such evidence may come from other update logic, documentation, tests, accounting semantics, or the original audit.
- **C3 (committed omission).** $\emptyset \subset \Delta_M(S, S') \subset M$, the writer creates an outstanding reconciliation obligation rather than merely beginning in an unrelated invalid state, and the required relation remains invalid at commit.

- **C4 (incorrect consumption).** Before reconciliation, a later transaction reads an omitted variable and the inconsistent joint state contributes to behavior that conflicts with the supported protocol obligation.

The writer and consumer are quantified over one coherent execution; separate feasible executions are insufficient. For protocols with lazy settlement, the relevant starting point may be a preceding synchronization boundary rather than a global invariant that holds at every intermediate step. The obligation may also depend on protocol phase or preceding transitions. The Kuiper fee-period rule, for example, cannot always be expressed as a predicate over only the current numerical supply and timestamp.

C4 distinguishes behavioral dependence from a defect. A read reaching require is evidence, not proof that the decision is wrong; the check may deliberately reject an inconsistent input. Conversely, stale state that incorrectly blocks an authorized action can be a genuine availability defect. We do not add an exploitability requirement, but validation must explain why the consumption conflicts with Φ .

3.2 Static evidence contract

MV-SCAN does not operate on concrete cells or synthesize Φ . It records abstract variables: scalar storage declarations, keyed variables such as $m[k_1] \cdots [k_j]$, and abstract external-state observations identified by receiver, selector, and arguments. An external observation is not automatically a persistent cell: a view may return a constant, an environmental value, or a combination of storage cells. Its persistence, mutability, and relevance require semantic review.

A relation hypothesis is $\rho = \langle M, \omega \rangle$, where M is a normalized set of eligible abstract variables and ω is the code origin that suggested their joint use. Construction guarantees at least two eligible descriptions, not two distinct concrete persistent cells. Mapping terms may alias, and an external observation may not denote a cell. Accordingly, C1 is a structural eligibility assumption in the bucket-level first-failure table, not a semantic theorem proved by candidate construction.

For a writer site w , the detector computes $\widehat{U}(w, \rho)$: the union of relation members independently matched by its abstraction to writes at that site. This is neither Δ_M nor necessarily a jointly realizable update set. For example, one symbolic write $m[k]$ may independently match $m[\emptyset]$ and $m[1]$ under different assumptions even though no one value of k updates both cells.

A location-specific read event is $e_r = \langle b_r, \ell \rangle$. The detector records $e_r \rightsquigarrow d$ when the read's abstract origin may influence a modeled sink d . Sinks comprise control operands, recorded value operands of modeled storage writes, and external effects. Destination-key-only dependence is not generally collected as a storage-write sink. May-influence supports review; it does not prove an incorrect decision or concrete outcome.

The resulting native candidate is writer-centered. It retains the hypothesis, writer, $\widehat{U}$, omitted variables, reader and sink evidence, abstract roots and storage domains, key constraints, and representative root-to-writer call-chain provenance. One static transaction root may represent many dynamic invocations with different inputs. The recorded chain is not guaranteed shortest, feasible, or a complete writer-to-reader trace.

3.3 Analysis objective

The static acceptance test requires: (i) a structurally eligible relation hypothesis; (ii) a nonempty proper subset under $\widehat{U}$; (iii) failure of a whole-root write-coverage heuristic that would otherwise filter the writer; (iv) a root-qualified omitted-variable read that may reach a modeled sink; and (v) compatibility under the represented storage, sender, argument, and key abstractions.

These conditions deliberately under- and over-approximate C1–C4. The whole-root filter records syntactic targets and is not a reconciliation proof: self-copies can remain in its generated sets, appropriate values are not checked, and coverage at return says nothing about an earlier same-transaction sink. We therefore call it a *whole-root write-coverage heuristic*, not a proof of a complete-group operation.

The objective is to preserve a compact relation–writer–omitted-consumer witness for an auditor. The detector does not prove concrete cell distinctness, protocol intent, a jointly feasible sequence, semantic value change, absence of reconciliation, incorrect behavior, or exploitability.

4 MV-SCAN

4.1 Overview

MV-SCAN is an annotation-free static may-analysis. It builds a state-annotated interprocedural CFG; infers relation hypotheses; summarizes write coverage and read-to-sink influence; enumerates context-compatible writers and consumers; and aggregates the surviving evidence around a writer-centered native candidate. The output is for review, not a proved bug.

4.2 Relation hypotheses

Control co-influence. For each modeled control sink, MV-SCAN collects abstract variables whose read-derived values may reach the condition. After eligibility filtering and normalization, two or more variables form an origin-specific hypothesis. Co-influence is useful because it identifies one decision made from several values; it is not independent evidence that those values must obey an invariant.

Return-derived relations. The second source is a return site in a function with no directly detected storage writes. This is not a transitive purity guarantee: the helper can call a mutating callee. One returned component must carry at least two variables; a tuple does not qualify merely because separate components expose one variable each. The analysis tracks components, but if a selected component has no recorded facts it falls back to tuple-wide information and can therefore mix components. The frozen environment option retains its historical “multi-return” name, while the paper uses the more accurate term *return-derived relation*.

Scalar storage, complete keyed variables, and abstract external-state observations are eligible. Constants and immutables are excluded, and a precise keyed variable shadows its coarse mapping base. Every hypothesis retains an *origin identity* (origin plus normalized members) and a member-only equivalence identity used for aggregation.

4.3 Writer evidence

At writer site w , the detector contextualizes each write description and independently matches it against relation members. After removing recognized syntactic self-copy patterns, the union of matches is $\widehat{U}(w, \rho)$. A site survives when $\emptyset \subset \widehat{U} \subset M$; the omitted set is $M \setminus \widehat{U}$.

$\widehat{U}$ is an abstract match set, not the semantic change set Δ_M , and its members need not be jointly realizable. A single $m[k]$ write can independently match $m[\emptyset]$ and $m[1]$ under different assumptions. The self-copy filter is also textual and lowercases identifiers, so Solidity names that differ only by case can be conflated.

Key matching preserves complete nested paths, substitutes formal arguments, and emits witness-local equality constraints when terms may coincide. The term language is deliberately restricted. State-variable references used as keys are treated as syntactically fixed even though their runtime values can change or coincide. Parameter canonicalization

uses single-origin provenance; a phi merge carrying one parameter origin is not proof that the merged value equals that parameter on every branch. External-return template substitution is not uniform across all variable kinds, so receiver and argument translation is not claimed complete.

The frozen detector then applies a *whole-root write-coverage heuristic*. It filters a writer when the root summary’s syntactic targets cover every relation member. Coverage is not reconciliation: the generated sets can retain self-copies, do not establish appropriate value changes, and do not prove the relation holds before a sink. This heuristic can therefore lose recall.

4.4 Consumer evidence

Read origins propagate through assignments, phi operations, references, tuple handling, resolved calls, and returns. Summaries record which parameters and physical read events may influence:

- control operands, including branches and require/assert;
- recorded value operands of modeled storage writes; and
- external effects.

Destination-key-only dependence is not generally recorded as a storage-write sink. An influence edge is behavioral evidence, not proof of a defect or changed concrete outcome.

Acceptance requires a location-specific read of an omitted variable under an owner-qualified root. Owner qualification remains mandatory even when the optional early root-context sink gate is disabled; the option controls an earlier filter, not the later multi-variable test. This is root sensitivity, not full callsite or invocation sensitivity.

4.5 Candidates and structural buckets

Native candidates are keyed by writer root, writer site, member-set equivalence, matched set, and omitted set. Reader witnesses retain roots, abstract storage domains, parameter bindings, senders, sink sites, and witness-local key constraints. Candidate-level constraints are a union for search and display; constraints from different witnesses are alternatives, not one conjunction.

The context-instance count is a projection over roots, storage domains, senders, and block identifiers. Argument-binding tuples remain in reader-witness records but are omitted from that count. Reported call provenance is a representative root-to-writer function chain chosen by the implementation’s parent selection. It is not guaranteed shortest, feasible, binding-exact, or a complete writer-to-reader execution trace.

For audit sampling, each native record maps to one structural payload containing dataset, subject, revision role, writer owner and block, relation members, matched and omitted members, transaction-context owners, sink kinds, origin sites, retained witness sites and their relation evidence, key constraints, and relation-shape tags. The native identifier, compilation-unit identity, and other details are discarded; identical payloads are deduplicated. A native record is not split into several buckets. Thus 3,732 B0 native records become 2,868 structural buckets through many-to-one merging. *Structural-bucket precision* is the fraction of audited buckets validated as MV-SI, not precision over native reports or distinct vulnerabilities.

4.6 Approximation and cost drivers

The analysis can over-approximate because a relation may be incidental, paths may not coexist, key constraints may be infeasible, or reconciliation may occur. It can under-approximate because relation origins, calls, keys, inheritance exposures, external observations, and whole-root coverage are incomplete abstractions.

The dominant costs are repeated summary propagation over IR, relation registration against access indexes, writer-reader Cartesian products, and contextual member matching. Call resolution and contexts are bounded, but fixed-point rounds, set sizes, and fan-out vary by program. We therefore report measured time and memory distributions instead of asserting an underspecified formal polynomial bound.

5 Implementation

We implement MV-SCAN as a Python detector integrated with Slither [15]. `inconsistent_state.py` controls relation registration, context expansion, witness checks, aggregation, and reporting. `icfg.py` provides the physical ICFG, variable abstractions, call resolution, context propagation, influence and write summaries, and matching. `mvscan_env.py` parses configuration. The detector code and evaluation runs are frozen; this section documents the measured implementation rather than an idealized algorithm.

5.1 Front end and contexts

Each declared function or modifier receives a source-qualified key, and each physical Slither CFG node is materialized once even when inheritance exposes it through several contracts. Direct calls resolve by canonical identity. Internal dispatch requires one concrete matching function in the inheritance lineage; library dispatch follows the library and signature. Optional interface dispatch is bounded. Excessive interface-dispatch fan-out leaves the call unresolved, whereas context overflow terminates the run.

Contexts track the external root, abstract contract storage domain, formal bindings, and active sender. The storage key uses source file and canonical contract declaration; it has no deployed-address or receiver-instance component. Different instances of one declaration can therefore share an abstract domain. One static root can represent many dynamic invocations.

The front end prefers Slither SSA IR and recovers complete nested mapping paths when possible. Formal parameters, senders, literals, state references, contracts, locals, and unknowns occupy separate term namespaces. Restricted parameter provenance can both create false matches and miss real ones: carrying one parameter-origin index through a merge does not prove equality on every branch. State-valued key terms are compared by syntactic identity even though their runtime values can change or coincide.

The no-op filter recognizes textual self-copy patterns and lowercases identifiers. It is therefore a heuristic, not semantic elimination. The storage-write sink collector records value influence on modeled operands and does not generally add dependencies carried only by a destination key.

5.2 Return and external observations

Return-derived hypotheses use functions with no directly detected storage writes, not functions proved transitively pure. One return component must carry at least two variables. Component information can fall back to tuple-wide facts when the selected component has none.

An unresolved high-level view result is represented as an abstract external-state observation identified by receiver, selector, and arguments. This identity does not establish a persistent cell; a view can return a constant, an environmental value, or a combination of cells, and can depend on caller context. Template substitution is not uniform across all observation kinds. View recognition also depends on the Slither attributes exposed by the pinned environment, so we claim only the behavior preserved in the frozen records.

5.3 Configuration and output

The reference configuration and four single-component ablations appear in Table 2. The historical environment-variable name for return-derived relations retains the term “multi-return.” Other settings control scalar witnesses, root differences, contextual keys, optional early root-specific sink filtering, dispatch and context bounds, root inclusion, entry-name normalization, and an optional secondary sink gate. The mandatory omitted-variable test remains root-qualified even when the optional early filter is disabled.

JSON output records versions, detector-module hashes, compiler and source digests, effective configuration, stage counts, relation and call catalogs, and deterministic candidate records. A candidate includes its writer, relation and origin sites, matched and omitted variables, projected context instances, reader witnesses, sink sites, representative call-chain provenance, and witness-local key constraints. Binding tuples remain in witness records but are omitted from the projected context count.

5.4 Frozen implementation limitations

The detector does not synthesize Φ , solve paths or transaction sequences, or prove exploitability. Its whole-root write-coverage filter is syntactic and can remove real omissions. Independently possible member matches can be unioned into an unrealizable full-write set. Calls, inheritance, keyed identity, external observations, and concrete deployment instances are approximated. The recorded call chain is selected lexically from available parents rather than by shortest-path distance and is neither a complete trace nor a feasibility proof. These are scope and validity limits; we do not propose detector changes or reinterpret the frozen results around a stronger implementation.

6 Evaluation Design

We evaluate MV-SCAN as an automatic static detector for a semantic property that existing smart-contract benchmarks do not label directly. The evaluation therefore separates historical-case recovery from output precision. Historical recovery asks whether a candidate reconstructs an already documented MV-SI end to end. Candidate precision asks how often a sampled detector-output unit satisfies the semantic criteria. The latter is not an open-world recall measurement and does not estimate MV-SI prevalence in all contracts.

6.1 Research questions

The evaluation addresses four questions:

- **RQ1—Historical recovery.** How many published inconsistent-state-update findings satisfy the MV-SI definition, and how often does the complete detector strictly recover the buildable cases?
- **RQ2—Candidate precision and component sensitivity.** What fraction of sampled structural buckets are MV-SI under the reference configuration and four single-component ablations, and how do the candidate populations change?
- **RQ3—Scalability and reproducibility.** How often do accepted projects complete, what time and memory do they require, and how stable is canonical output across paired seeds?
- **RQ4—Error concentration.** At which semantic criterion do nonbugs fail, and which implementation or program behaviors account for those failures?

Table 1. Benchmark populations and best-effort no-edit build coverage.

Benchmark	Population	Resolved	Built	Excluded	Coverage
Published ISU findings	116	116	106	10	91.4 %
Web3Bugs snapshots	102	102	65	37	63.7 %

We do not use adjacent tools as a head-to-head accuracy or throughput baseline. Their target properties, required inputs, report units, corpora, and validation procedures differ. [section 10](#) instead positions them qualitatively by the evidence they are designed to produce.

6.2 Benchmarks and build policy

The first benchmark is the complete set of 116 published inconsistent-state-update findings assembled by Li et al. [23]. The second is the 102 archived project snapshots in Web3Bugs [43]. For both populations, we resolve the referenced source revision and apply a best-effort no-edit build policy: dependencies and historical toolchains may be restored, but the evaluated Solidity source is not rewritten merely to make the detector run. A resolved item is marked built only when the accepted compilation artifacts support the analysis. [Table 1](#) reports the resulting coverage; exclusions remain outside denominators that require executable detector output.

The unit of the historical benchmark is a published finding. Several findings may map to the same repository or source revision, whereas runtime is measured per accepted analysis run. The unit of the Web3Bugs build population is a project snapshot. We preserve these distinct units rather than converting project-level executions into per-contract rates.

6.3 Historical MV-SI oracle and strict matching

Two authors independently classify each of the 116 published findings under C1–C4 from [subsection 3.1](#); disagreements are then adjudicated through discussion. The source report can therefore receive one of three final labels: MV-SI, non-MVSI, or insufficient evidence. The last label is used when the available report, source, and reproduction material do not support a reliable decision; it is not silently counted as either positive or negative. Initial labels are recorded before adjudication so that disagreement remains visible as evidence of the construct’s semantic difficulty.

Only adjudicated MV-SI findings with an accepted build enter the strict-recovery denominator. A MV-SCAN candidate is a strict match when the evidence corresponds to the same documented failure rather than merely touching one involved variable. In particular, the match must recover (i) the relevant coupled relation or a directly specialized keyed instance, (ii) the writer transition that updates the documented proper subset, (iii) at least one documented omitted member, and (iv) a reader/sink role equivalent to the behavior described by the finding. A variable-level localization, adjacent relation, or warning in the same function is not a strict recovery.

This protocol measures a lower bound on end-to-end recovery over known cases. It does not imply that every historical report contains enough detail for a unique oracle, nor that a nonmatching candidate has no diagnostic value. The separate insufficient-evidence class and the strict semantic rule prevent those ambiguities from inflating recall.

6.4 Configurations

[Table 2](#) defines the frozen reference and four component ablations. Each ablation changes one named relation or identity mechanism from B0; all other evaluation settings and the build population remain fixed. The experiment is a

Table 2. Reference configuration and component ablations.

ID	Change from B0	Mechanism examined
B0	None	Complete frozen detector
A1	Branch-derived relations disabled	Control-derived relation inference
A2	Multi-return relations disabled	Multi-return relation inference
A4	Mapping-insensitive state identity	Precise mapping-location identity
A5	Contextual keys disabled	Key, storage, sender, and argument context

configuration-sensitivity study rather than a monotone feature-addition sequence. Removing a component can delete candidates, split or merge candidate identities, change the sampled population, and alter which semantic cases are represented.

6.5 Native candidates, structural buckets, and sampling

The detector’s native JSON unit is the writer-centered candidate defined in [subsection 4.5](#). The frozen bucketing script maps each native record to one structural payload and deduplicates identical payloads; it does not split a native record by reader or sink. The payload contains dataset, subject, revision role, writer owner and block, relation members, matched and omitted members, transaction-context owners, sink kinds, origin sites, retained witness sites and relation evidence, key constraints, and shape tags. This exact projection discards native IDs and compilation-unit identity, allowing multiple native records to merge.

For each configuration, we draw an equal-probability sample of 400 distinct structural buckets without replacement and audit every selected bucket under C1–C4. The label is positive only when all criteria hold. Structural-bucket precision is

$$\hat{p} = \frac{\text{\#audited structural buckets labeled MV-SI}}{\text{\#audited structural buckets}}.$$

We report ordinary Wilson 95% intervals as binomial approximations for each 400-bucket sample. They do not apply a finite-population correction, which matters most for the 400-of-550 A1 sample. Since samples are drawn separately from configuration-specific populations, differences are descriptive rather than paired treatment effects.

To interpret population changes, we report both all buckets containing each configuration and buckets unique to that configuration. The latter identify evidence introduced only under one analysis setting, but they still do not isolate a causal component effect because disabling a mechanism can change relation and candidate identities upstream.

6.6 Error analysis

For every distinct audited bucket that does not satisfy MV-SI, reviewers record the first failed semantic criterion. The frozen form begins at C2 because construction enforces structural eligibility; C1 is therefore an assumption of this table, not a proved fact about distinct concrete cells. Each nonbug also receives one mutually exclusive primary cause. The causes distinguish, for example, a writer that never produces a partial persistent state from one that reconciles before consumption.

Sampled buckets receive a binary final label. When review cannot establish a feasible MV-SI, the bucket is counted negative; the metric is therefore a strict confirmation fraction, not proof that every negative is infeasible. Relation support is measured independently of final candidate validity. A bucket can contain a well-supported relation and still fail C3 or C4.

Table 3. Historical ISU classification and strict semantic recovery.

Outcome	Count	Share or rate
Initial reviewer disagreements	52	44.8 % of 116
Adjudicated MV-SI	64	55.2 % of 116
Non-MVSI	39	33.6 % of 116
Insufficient evidence	13	11.2 % of 116
MV-SI with accepted build	62	96.9 % of 64
Strict matches	13	21.0 % of 62

6.7 Scalability and deterministic reproducibility

For each accepted analysis run and configuration, the evaluation records completion, wall-clock time, and peak resident-set size. We report medians, interquartile ranges, and maxima rather than only means because repository costs are strongly skewed. We make no cross-tool speedup claim and do not normalize the measurements to seconds per contract.

Deterministic reproducibility is assessed by comparing canonical output from available paired seeds under B0. Equality is based on canonicalized detector output rather than raw object order. Missing pairs are reported separately from mismatched pairs, and the evaluation records whether any run exhausted time, memory, or the explicit context bound.

7 Results

7.1 RQ1: Historical recovery

The historical classification confirms that MV-SI is a narrower target than the published ISU population. Of 116 findings, reviewers adjudicate 64 (55.2%) as MV-SI, 39 (33.6%) as non-MVSI, and 13 (11.2%) as having insufficient evidence. The 52 initial disagreements, or 44.8% of the population, show that even with a written criterion set, deciding whether a reported update error is relational, persistent, and behaviorally consumed requires substantial semantic judgment.

Build coverage is high within the positive class: 62 of 64 MV-SI findings have an accepted build. MV-SCAN strictly matches 13 of those 62 cases, yielding 21.0% end-to-end recovery. The result should not be read as variable localization. A strict match requires the relation, the relevant partial writer, an omitted member, and the corresponding consumer role in one candidate. The remaining 49 buildable cases include both complete misses and nearby evidence that does not satisfy that strict rule. We did not classify these misses by cause and therefore do not infer a false-negative taxonomy.

Answer to RQ1. The complete detector recovers roughly one fifth of buildable, adjudicated historical MV-SI cases under a strict semantic oracle. This establishes nontrivial coverage while leaving substantial room for relation origins and transition patterns not represented by the current static model.

7.2 RQ2: Candidate precision and component sensitivity

Under B0, 82 of 400 audited structural buckets satisfy C1–C4, for 20.5% structural-bucket precision with an ordinary Wilson 95% interval of 16.8–24.7%. The four ablations range from 18.2% to 28.5%. A1, which disables branch-derived relations, has the highest sampled precision at 28.5%; A2, which disables return-derived relations, has the lowest at 18.2%. The intervals are binomial approximations without a finite-population correction, and the samples are not paired, so the ranking is descriptive.

Table 4. Structural-bucket precision by configuration.

Configuration	Audited buckets	MV-SI buckets	Precision (95% CI)
B0	400	82	20.5% [16.8%, 24.7%]
A1	400	114	28.5% [24.3%, 33.1%]
A2	400	73	18.2% [14.8%, 22.3%]
A4	400	94	23.5% [19.6%, 27.9%]
A5	400	77	19.2% [15.7%, 23.4%]

Table 5. Candidate populations and configuration-specific structural buckets.

Config.	All buckets containing configuration			Configuration-only buckets			
	Native	Structural	Audited	Population	Audited	MV-SI	Δ precision
B0	3,732	2,868	400	154	24	5	0.0 pp
A1	688	550	400	324	237	53	+8.0 pp
A2	3,370	2,636	400	326	48	10	-2.2 pp
A4	4,735	3,502	400	2,100	251	70	+3.0 pp
A5	3,725	3,007	400	1,254	171	33	-1.2 pp

Note: The frozen cross-configuration union contains 6,873 structural buckets. The five samples contain 2,000 selections and 1,270 distinct audited buckets; 376 buckets occur in more than one configuration sample. Precision deltas compare separate equal-probability samples and are descriptive, not paired effects.

The population changes are larger than the precision changes. B0 emits 3,732 native candidates, which project to 2,868 structural buckets. Disabling branch-derived relations reduces that structural population to 550, an 80.8% reduction, while retaining a denser sampled subset. This result indicates that control-derived relations are the main coverage-expanding relation source in the frozen corpus; it does not show that those relations are generally invalid. Indeed, the error analysis below finds unsupported relations to be rare across the audited union.

Disabling return-derived relations yields 2,636 structural buckets, close to B0, but the frozen table reports 326 buckets unique to A2. The origin-insensitive native key and independently computed return-location summaries do not explain this non-monotonicity. A downstream projection difference, mismatched run inputs, or upstream nondeterminism could do so, but the preserved records do not establish which; we retain the count and treat its provenance as unresolved. Mapping-insensitive A4 expands the population to 3,502 buckets, including 2,100 configuration-only buckets. Disabling contextual keys in A5 produces 3,007 buckets, 1,254 configuration-only.

Across all configurations, the frozen union contains 6,873 structural buckets. The five samples contain 2,000 selections and 1,270 distinct audited buckets. Direct reconciliation of the frozen membership sets shows that 376 buckets were selected in more than one configuration: 143 appear twice, 112 three times, and 121 four times. The previously reported value 156 did not describe this overlap and is removed. Precision deltas compare separate samples, not paired counterfactuals.

Answer to RQ2. The reference detector yields 20.5% structural-bucket precision under strict MV-SI labeling. Relation and identity mechanisms substantially reshape the candidate population; branch-derived relations provide the largest expansion, while precise mapping and contextual keys prevent many alternative groupings. Sample precision alone is insufficient to rank the mechanisms because each setting changes the population being sampled.

Table 6. Completion, scalability, and deterministic reproducibility.

Benchmark	Config.	Success	Median time	Time IQR	Max time	Median RSS	Max RSS
ISU	B0	88/88	12.90 s	5.80–23.03 s	659.39 s	332.9 MiB	15,534.9 MiB
	A1	88/88	11.42 s	5.61–21.58 s	316.72 s	322.5 MiB	5,167.7 MiB
	A2	88/88	12.67 s	5.80–22.90 s	396.66 s	321.1 MiB	8,327.4 MiB
	A4	88/88	14.73 s	5.96–22.65 s	420.74 s	340.3 MiB	9,155.9 MiB
	A5	88/88	12.82 s	5.78–21.89 s	502.09 s	331.2 MiB	12,529.7 MiB
Web3Bugs	B0	61/65	7.61 s	3.84–20.99 s	59.58 s	237.0 MiB	1,047.4 MiB
	A1	61/65	7.31 s	3.74–14.78 s	58.32 s	229.2 MiB	1,044.1 MiB
	A2	61/65	7.60 s	3.84–18.49 s	59.72 s	231.4 MiB	1,046.8 MiB
	A4	61/65	8.06 s	3.84–20.90 s	64.88 s	244.5 MiB	1,048.5 MiB
	A5	61/65	7.61 s	3.79–20.20 s	59.72 s	233.1 MiB	1,043.0 MiB

Determinism: For B0, 76 of 80 available ISU seed pairs and all 61 Web3Bugs seed pairs had identical canonical output; four ISU pairs mismatched. Eight ISU and four Web3Bugs pairs were missing. No run timed out, exhausted memory, or reached the context bound.

7.3 RQ3: Scalability and deterministic reproducibility

All 88 ISU analysis runs complete under every configuration. On Web3Bugs, 61 of 65 accepted snapshots produce successful results under every configuration. The measurement record reports no timeout, memory exhaustion, or context-bound event; the frozen table therefore does not attribute the four unsuccessful snapshots to any of those resource failures.

For B0, the median Web3Bugs runtime is 7.61 seconds, with an interquartile range of 3.84–20.99 seconds and a maximum of 59.58 seconds. Median peak RSS is 237.0 MiB and the maximum is 1,047.4 MiB. The ISU runs are heavier: median time is 12.90 seconds, but the maximum reaches 659.39 seconds; median RSS is 332.9 MiB and the maximum reaches 15,534.9 MiB. The medians support repository-scale triage, while the long ISU tail rules out a blanket claim that every run is low-cost.

The ablations have similar medians. On Web3Bugs, median times range from 7.31 to 8.06 seconds and median RSS from 229.2 to 244.5 MiB. On ISU, median times range from 11.42 to 14.73 seconds and median RSS from 321.1 to 340.3 MiB. The largest tail costs vary more substantially. Because the mechanisms change the number and shape of propagated facts, the table is best interpreted as an empirical resource profile rather than an asymptotic ranking.

Canonical output is identical for all 61 available Web3Bugs B0 seed pairs and for 76 of 80 available ISU pairs. Four ISU pairs mismatch; eight ISU and four Web3Bugs pairs are missing. The detector is therefore highly, but not perfectly, reproducible under the measured environment. The mismatches matter because candidate identities, contexts, and relation catalogs should ideally be independent of incidental front-end ordering.

Answer to RQ3. Most accepted projects complete in seconds with hundreds of MiB at the median, including every ISU run and 61 of 65 Web3Bugs snapshots. A small heavy tail and four canonical-output mismatches remain important engineering limitations.

7.4 RQ4: Error concentration

The 1,270 distinct audited buckets contain 969 nonbugs and therefore 301 buckets that satisfy the MV-SI criteria. Reviewers support the proposed relation in 1,239 buckets, or 97.6%; this bucket-weighted conditional rate is not a census of distinct relations. Only 31 buckets first fail C2. Only one bucket first fails C4, but first-failure coding sensors C4 among every C2- or C3-negative bucket. The table therefore does not independently measure consumer-gate effectiveness.

Table 7. Where audited Web3Bugs candidates fail semantic validation.

Outcome	Count	Share
<i>First failed criterion (all 1,270 audited buckets)</i>		
C2: unsupported relation	31	2.4% of all
C3: no persistent partial transition	937	73.8% of all
C4: no behavioral consumption	1	0.1% of all
<i>Primary cause (969 nonbugs)</i>		
No desynchronizing partial transition	522	53.9%
Reconciliation precedes consumption	291	30.0%
Analysis abstraction error	104	10.7%
Unsupported or overbroad relation	31	3.2%
Incompatible context, key, or ordering	20	2.1%
Omitted state not behaviorally consumed	1	0.1%

Note: Reviewers judged 1,239 of 1,270 inferred relations (97.6%) to be supported by protocol semantics. The first two primary causes account for 813 of 969 nonbugs (83.9%).

The primary-cause labels explain why. In 522 nonbugs, the candidate’s writer evidence does not correspond to a desynchronizing partial transition. In another 291, reconciliation precedes the relevant consumption. These two causes total 813 of 969 nonbugs (83.9%). The preserved summary does not establish whether every reconciliation occurred before writer commit or later before use.

The remaining causes are smaller but actionable. Analysis abstraction errors account for 104 nonbugs, and incompatible context, key, or ordering accounts for 20. These categories reflect the cost of may-reachability, bounded call resolution, key unification, and witness combination. Unsupported or overbroad relations account for 31, matching the C2 first-failure count. One nonbug contains an omitted state member that does not behaviorally reach the claimed use.

Answer to RQ4. Among reported buckets, the current bottleneck is transition semantics. The detector usually finds a defensible relation and recorded consumer evidence, but often cannot distinguish a persistent proper-subset transition from a local write that is impossible, non-desynchronizing, or reconciled before use.

7.5 Summary of findings

Together, the results establish a clear evidence hierarchy. Annotation-free relation recovery is viable: relations are semantically supported in nearly all audited buckets. The complete pipeline nevertheless achieves 20.5% sampled precision and 21.0% strict historical recovery because relation support alone is far weaker than an end-to-end MV-SI. The gap concentrates in one place—proving that the writer can commit only part of the relation and that no reconciliation occurs before the omitted member affects behavior. [section 8](#) develops the implications of that concentration.

8 Discussion and Practical Implications

8.1 Relation discovery is feasible, but relation discovery is not detection

The strongest positive result is the 97.6% semantic support rate for inferred relations. It answers the initial feasibility question behind MV-SCAN: useful multi-variable structure can be recovered from code without project annotations or a mined natural-language specification. Control decisions and state-returning APIs encode enough of the developer’s model to produce credible member sets across a large and heterogeneous corpus.

That result should not be conflated with candidate precision. A relation can be real while a particular writer is harmless. In fact, most false positives contain exactly this shape: the protocol genuinely relates the locations, and an omitted member genuinely reaches a behavioral sink, but the alleged writer does not commit an inconsistent state. Relation inference solves the absence of an aliasing anchor; it does not solve path feasibility, transactional completion, or reconciliation.

This distinction changes the research agenda. A detector that responds to 20.5% precision by mining ever narrower relations risks deleting useful coverage while leaving the dominant error untouched. The audit suggests a better decomposition: retain broad but provenance-rich relation hypotheses, then spend precision budget on transition summaries that answer whether the relation can remain partially updated at an observable boundary.

8.2 Persistent-transition modeling is the central bottleneck

The C3 failures reveal two different analytical problems. The first is *desynchronization*: a block writes one member, but the transaction does not actually change the relation in the relevant way. The write may assign an equivalent value, operate on a different logical account, occur only on an infeasible branch, or be dominated by conditions that preserve the relation. The second is *reconciliation*: the transaction temporarily writes a proper subset, but another mandatory operation restores the relation before a consumer can observe it.

The whole-root write-coverage heuristic addresses only a coarse subset of reconciliation. It removes a writer when the root’s syntactic targets cover all relation variables, but it does not prove effective changes, compatible values, or restoration of the obligation. Conversely, self-copies and independently possible key matches can make coverage look complete and lose recall.

A natural next analysis is a relation-specific transaction summary. Rather than summarize only which locations a root may or must write, such a summary would preserve ordered phases: the first relation member changed, the set known synchronized at each exit, and key substitutions under which the synchronization holds. Post-dominance and path-sensitive must analysis could discharge common cases without invoking a solver. A targeted solver could then be reserved for candidates whose static summary leaves one or two concrete path or key obligations, instead of exploring arbitrary transaction sequences from scratch.

The distinction also matters to manual review. Auditors should not begin by re-litigating whether the relation is plausible. They should trace the writer root to its non-reverting exits and ask a narrower question: after all internal calls, modifiers, and deferred accounting complete, can any omitted member remain semantically stale for the same key? This ordering makes review faster because it follows the empirical failure distribution.

8.3 What the ablations do and do not show

Disabling branch-derived relations removes 80.8% of B0’s structural population and raises sample precision by 8.0 percentage points. The most defensible interpretation is that return-derived relation candidates are denser in the sampled corpus, while branch-derived relations provide much broader coverage. It would be incorrect to conclude that control-derived relations are mostly unsupported: only 2.4% of all distinct audited buckets first fail the relation criterion. More likely, broad control relations expose many genuine couplings whose apparent writers are not persistent partial transitions.

The result creates a practical operating choice. An auditor with a tight review budget may begin with return-derived candidates or rank branch relations lower. A benchmark or discovery campaign that values coverage should retain branch relations but apply stronger writer-side prioritization. The complete configuration is therefore not justified

because every component maximizes point precision; it is justified because it preserves the full detection hypothesis under study.

The mapping and contextual-key ablations require even more caution. A4 and A5 do not merely relax a final filter. They alter the identity of state locations and the way evidence aggregates, creating thousands of configuration-only structural buckets. Their modest sample-precision changes cannot capture the semantic value of avoiding a false equality such as `balance[alice] = balance[bob]` or of separating the same implementation under two deployed storage contexts. For a reviewer, those distinctions are non-negotiable even when a coarser configuration happens to sample a denser set.

Disabling return-derived relations slightly lowers sampled precision but leaves a large structural population. The 326 A2-only buckets are not explained by the documented origin-insensitive aggregation, so we do not assign that non-monotonicity a cause without stronger provenance evidence.

8.4 A practical triage workflow

The writer-centered output supports a disciplined review sequence:

- (1) **Validate the relation origin.** Inspect the control sink or return site and state why the members participate in one protocol rule. Reject incidental conjunctions immediately.
- (2) **Select one concrete witness.** Use one writer context, reader context, and its local key constraints. Do not conjoin the candidate-wide union of constraints.
- (3) **Establish the effective writer transition.** Follow the outer writer root through modifiers and calls; determine which relation members change for the same logical key and whether their new values can violate the relation.
- (4) **Search for mandatory reconciliation.** Check post-dominating writes, settlement helpers, hooks, and exit-specific clean-up. A block-level omission is not sufficient.
- (5) **Validate the consumer.** Confirm that the omitted member reaches the reported control, state write, or external effect before any synchronization and that the inconsistent value can change behavior.
- (6) **Assess impact separately.** Only after C1–C4 hold should review proceed to exploit preconditions, privileges, liquidity, ordering, or economic loss.

This sequence mirrors the evidence already serialized by MV-SCAN and the observed false-positive distribution. It also avoids a common reporting error: calling every supported relation plus partial-looking assignment an MV-SI before checking transaction completion.

For continuous integration, a lower-volume profile can prioritize candidates with a small relation, concrete mapping keys, one writer root, a control or external-effect sink, and no observed reconciliation write in the root summary. For security review, the full configuration is more appropriate because it preserves branch-derived relations and context metadata. These are ranking strategies over the same candidate semantics, not alternate definitions of MV-SI.

8.5 Implications for automated analysis

First, multi-variable analysis benefits from separating *relation evidence* from *violation evidence*. The former can be mined broadly and audited independently; the latter requires transition and consumer reasoning. Combining them into one opaque score would make it impossible to see that relation support is already high while writer validity is low.

Second, state identity is part of the semantic model, not an implementation detail. Complete mapping-key paths, deployment storage, formal substitutions, and active-sender changes determine whether two accesses refer to one

logical fact. The large configuration-only populations in A4 and A5 show how quickly candidate structure changes when these identities are weakened.

Third, consumer gating remains valuable even though only one final nonbug fails C4. That low count does not mean sink analysis is unnecessary; it means the pipeline filters out nonbehavioral reads before sampling and reports candidates with explicit consumer evidence. Removing the gate would change the candidate universe. The error table characterizes survivors, not every coarse access pair considered internally.

Fourth, a hybrid design is most promising after static narrowing. Symbolic execution supplied with one relation, one writer root, one reader root, and concrete witness-local equality constraints faces a different problem from unconstrained whole-program exploration. The static analysis can therefore serve as a specification and target generator for selective feasibility checking. Any future hybrid evaluation should compare that complete workflow on the same cases and report the additional cases proved, rejected, and timed out; published runtimes from unrelated detectors are not an adequate baseline.

8.6 Scope relative to adjacent tools

MV-SCAN does not replace same-state race, reentrancy, transaction-ordering, symbolic-execution, or general static-analysis tools. Those systems answer complementary questions and may provide the mechanism through which an MV-SI is exposed. A transaction-ordering detector can show that two calls may be reordered; a symbolic executor can validate a concrete sequence; a general framework can expose access control or call hazards. MV-SCAN contributes the relation-partial-transition-consumer hypothesis that tells those analyses which distinct locations should be reasoned about together.

Accordingly, the meaningful comparison is qualitative. Does a tool infer or accept a multi-location relation? Does it represent a proper-subset transition? Does it require behaviorally consumed omitted state? Does it prove feasibility or merely triage? Comparing raw warning counts, per-contract runtimes, or precision values across different answers to these questions would obscure rather than clarify progress.

8.7 Resource profile and deployment

The runtime medians support use during repository-scale audits and repeated experimentation. The heavy ISU tail nevertheless matters operationally. A 15,534.9-MiB maximum does not by itself invalidate the characterization of MV-SCAN as a static, scalable analysis; it does show that resource provisioning and tail monitoring are necessary. CI deployments should impose transparent process limits, persist partial metadata, and report whether a project was missing, failed, or completed with no candidates. Research evaluations should continue reporting distributions and maxima rather than converting total time into a deceptively uniform per-contract rate.

The four ISU determinism mismatches deserve similar treatment. Stable sorting and structural digests already reduce incidental nondeterminism, but compiler, Slither, dependency, or set-order effects may still alter upstream facts. Reproducibility should be treated as a measured property of the complete toolchain, not assumed from deterministic candidate hashing alone.

9 Limitations and Threats to Validity

9.1 Construct validity

MV-SI is semantic. C1–C4 reduce ambiguity, but C2 and C3 still require protocol understanding. A relation may be documented incompletely, and whether a partial state is erroneous can depend on economic intent not recoverable from source alone. The 52 initial disagreements among 116 historical findings quantify this threat rather than eliminating it. Adjudication produces a consistent final oracle, but it does not make the construct objective in the way a compiler error is objective.

Structural-bucket precision. The 20.5% reference result is the fraction of sampled structural buckets labeled MV-SI. It is not raw-report precision, native writer-candidate precision, per-project precision, vulnerability prevalence, or the probability that a random DeFi contract contains MV-SI. Structural bucketing can split one underlying defect across several readers or sinks, while native aggregation can combine several witnesses under one writer. We state the unit with every metric, but no one normalization is uniquely correct for auditor effort.

Strict recovery is intentionally narrow. A historical case counts only when one candidate matches the relation, writer, omitted state, and consumer role. This rule avoids inflating recall with adjacent variable warnings, but can count diagnostically useful near-matches as misses. Conversely, historical reports may omit enough detail that the adjudicated relation or sink is only one plausible reconstruction. The 13 insufficient-evidence findings are excluded rather than forced into the oracle, yet uncertainty may remain among retained cases.

Behavioral sinks are proxies. Control conditions, storage writes, and external effects are useful points where stale data can matter, but they are not equivalent to economic harm. A value can reach a sink without changing the concrete outcome, and an economically important use may occur through a pattern not recognized by the sink classifier. MV-SCAN therefore reports behavioral influence, not exploitability.

9.2 Internal validity

No joint path-feasibility proof. The detector combines may-reachable writer and reader evidence under compatible contexts. It does not prove that one initial state and transaction sequence realizes all facts, that witness-local key equalities hold together, or that the reader precedes every reconciliation. This is the largest known source of imprecision: 813 of 969 nonbugs either lack a desynchronizing partial transition or reconcile before consumption.

Whole-root write coverage. The filter uses syntactic root-level storage targets, not a reconciliation proof. Self-copies can remain in its generated sets, independent key matches can form an unrealizable full set, and values are not checked. It can therefore remove real omissions as well as retain harmless writers.

State and context abstraction. Mapping keys are canonicalized syntactically and unified through a restricted term language. Equal runtime expressions with different syntax can be missed, while free transaction terms may produce equality obligations that are infeasible. Context propagation distinguishes roots, deployment storage, formal arguments, and active senders, but is bounded and does not model arbitrary delegatecall/proxy behavior. External state covers unresolved view results, not the complete mutable state of remote contracts.

Call and inheritance resolution. Internal and library calls generally require a unique statically resolved target; interface dispatch is optional and capped. Physical inherited implementations are not cloned for every exposure, and owner-specific summaries can under-approximate some inherited roots. These choices can create false negatives and can also remove reconciliation facts, indirectly creating false positives.

Front-end and build dependence. The analysis depends on Slither’s source mapping, CFG, SSA IR, reference recovery, and call objects [15]. Historical repositories depend on old compilers and packages. A best-effort no-edit build policy prevents source repair from contaminating the subject, but accepted builds may still differ in dependency environment from the original audit. Four canonical-output mismatches among the available ISU seed pairs show that the complete toolchain is not perfectly deterministic.

Manual labeling. The final labels are based on human inspection. Initial disagreement is reported for the historical set, but the frozen tables do not provide a separate inter-rater coefficient for every Web3Bugs bucket packet. Adjudication and preserved evidence reduce accidental inconsistency, yet reviewer expectations about intended synchronization can still bias labels. The same reviewers also know that buckets were generated by MV-SCAN, which can anchor attention on the proposed relation.

9.3 External validity

Benchmark coverage. The historical population covers 116 published ISU findings, but only 64 are adjudicated MV-SI and 62 of those build. Web3Bugs provides realistic repository snapshots and dependency context, yet only 65 of 102 snapshots receive accepted builds and 61 complete under each configuration. Results may not generalize to unbuildable projects, newer Solidity features, proxy-heavy deployments, non-EVM platforms, closed-source contracts, or protocols whose invariants live primarily in documentation.

Corpus composition. Both benchmarks are security-oriented and concentrate on audited or reported DeFi code. Their relation density, coding style, and defect distribution may differ from ordinary contracts or other domains. The 97.6% relation-support rate should therefore be read as evidence that the chosen signals work in these corpora, not as a universal rate for all Solidity programs.

Sampling and configuration comparisons. Each configuration contributes an equal-probability sample of 400 structural buckets, not a census of its full population. Wilson intervals quantify binomial sampling uncertainty within each sample, but the samples overlap and are not paired. Differences such as A1’s +8.0 percentage points cannot be isolated from population and identity changes. Configuration-only buckets further show that ablations are not monotone subsets.

No open-world recall. Web3Bugs is used for detector-output precision, not as a complete negative-and-positive oracle for MV-SI. Unreported MV-SI cases may exist outside sampled output, and unsampled candidates are unlabeled. Strict recall is reported only for the adjudicated historical positive set.

9.4 Resource and reproducibility threats

Median runtimes and memory are practical, but the ISU maxima of 659.39 seconds and 15,534.9 MiB reveal a heavy tail. Hardware, compiler caching, filesystem behavior, and Slither version can affect these measurements. Because the evaluation does not compare matched targets on identical corpora, the values should not be converted into cross-tool speedups.

Canonical-output equality holds for all available Web3Bugs seed pairs and 76 of 80 available ISU pairs, with four mismatches and additional missing pairs. Candidate hashing and deterministic sorting do not guarantee deterministic upstream analysis. Future releases should preserve exact environments and investigate each mismatch before treating the output digest as a reproducibility oracle.

9.5 Claims deliberately not made

The evaluation does not establish that MV-SCAN proves vulnerabilities, discovers a particular number of zero-days, outperforms adjacent detectors in precision or recall, or is uniformly lightweight. It establishes strict recovery on one historical MV-SI oracle, structural-bucket precision on sampled output, component-sensitive population changes, measured resource distributions, and a concentrated error profile.

10 Related Work

10.1 Inconsistent updates and multi-variable correlations

MUVI established that access correlations among distinct variables can expose semantic and concurrency bugs even when the variables have no aliasing relation [26]. Its central observation—that program logic often reveals which variables should be reasoned about together—is also the starting point for MV-SCAN. The target and setting differ: MUVI studies correlated accesses in systems software, whereas MV-SCAN constructs origin- and key-qualified persistent-state relations and asks whether an omitted member reaches a behavioral consumer in Solidity.

Li et al. provide the closest empirical account of inconsistent state update (ISU) vulnerabilities in smart contracts [23]. Their study organizes 116 real-world findings by root cause, fixes, and exploitation method and demonstrates a proof-of-concept checker for one local symptom. We use the full finding population as the source of the historical classification in section 6. MV-SI is not proposed as a replacement for ISU. It is an operational subset: at least two persistent locations participate in a supported relation, a feasible transition commits a proper-subset update, and omitted state is behaviorally consumed before reconciliation. This restriction makes relation-plus-transition-plus-consumer candidates measurable, while excluding many ordinary initialization, single-state, or incorrect-update cases within the broader ISU family.

10.2 Automatic invariant and property generation

Several systems automatically generate contract-specific properties, so MV-Scan’s contribution is not the first automatic inference of cross-variable relations. InvCon+ combines dynamic inference with static verification and includes mapping-sum, supply, and pre/postcondition relations [4]. SmartInv uses code and natural-language context to infer invariants that can contain old values, mapping sums, and cross-function or cross-contract properties [3]. SmartOracle mines fine-grained likely invariants from transaction histories, while Trace2Inv instantiates guards from templates and historical executions [1, 6]. PropertyGPT generates properties through retrieval and checks them with a prover; ScType targets accounting errors with a type-based analysis [2, 5].

These systems differ in inputs, property languages, and validation obligations. MV-Scan contributes a narrower witness structure assembled without project annotations or transaction traces: a relation hypothesis, an abstract proper-subset writer, and an omitted-variable consumer. We do not compare precision numbers across unmatched corpora or claim that neighboring systems cannot find a defect that also satisfies MV-SI.

10.3 Same-state inconsistency, reentrancy, and transaction ordering

A large body of smart-contract analysis starts from conflicting accesses to one storage location. Reentrancy analyses identify executions in which an external call permits a state-dependent operation to be re-entered before an expected update [8, 11, 27, 32, 41]. Transaction-ordering analyses reason about outcomes that depend on how independently submitted calls are sequenced [21, 28, 30, 38, 42]. These mechanisms can enable or expose an MV-SI, but they do not by themselves identify which distinct state locations are required to remain related.

SAILFISH uses storage-dependency reasoning to vet state-inconsistency bugs efficiently [7]. DIVERTSCAN combines flow-divergence analysis with multiplex symbolic execution to determine whether a same-state conflict can reach critical behavior from a common starting state [25]. Its distinction between a state conflict and a consequential use motivates MV-SCAN’s consumer condition, and the frozen implementation adapts parts of its self-copy and sink logic. These analyses begin from a shared-variable or race candidate, whereas MV-SI constructs a relation–writer–consumer report unit. This anchor distinction does not mean DivertScan categorically cannot find a defect that also satisfies MV-SI.

10.4 General static analysis and formal reasoning for smart contracts

Slither provides the source-level CFG, SSA IR, storage-access facts, and detector interface on which MV-SCAN is implemented [15]. Other static systems encode syntactic checks, semantic facts, Datalog rules, bytecode dependencies, or composite vulnerability patterns, including SmartCheck, Securify, SmartDagger, Vandal, and Ethainter [8, 9, 24, 35, 37]. Zeus and related verification systems translate contracts into intermediate verification conditions [20]. These frameworks can express or host checks over multiple variables when supplied with an appropriate rule or specification. MV-SCAN contributes a particular automatic source of that specification: code-derived relation hypotheses paired with a proper-subset writer and omitted-member consumer.

Symbolic execution and bounded model checking explore concrete or symbolic paths and can validate a supplied vulnerability hypothesis. Mythril, Manticore, TeEther, EthBMC, and summary-based symbolic evaluation represent different points in this design space [14, 16, 17, 22, 29]. Solver-backed storage arrays and keccak-derived slot expressions can reason more precisely about symbolic keys than MV-SCAN’s bounded term unification. In return, MV-SCAN avoids whole-program path exploration and emits target-specific constraints that could seed a later selective solver. The two approaches are complementary: static relation and consumer discovery narrows the question; symbolic reasoning can attempt to prove one writer–reader sequence.

10.5 Fuzzing and learned exploration

ContractFuzzer, sFuzz, Echidna, Harvey, SMARTIAN, and SmarTest use randomized, feedback-directed, data-flow-guided, or learned exploration to reach contract failures [12, 19, 31, 33, 36, 39]. Targeted greybox techniques improve the chance of reaching selected program points or transaction sequences [40]. Dynamic exploration is well suited to validating concrete properties, but an MV-SI may not revert or violate a generic runtime oracle: every local operation can succeed while the joint state is economically wrong. A relation or exploit predicate is therefore still required. MV-SCAN can supply candidate relations, writers, omitted members, and sinks as targets for property-based or directed testing; it does not claim to replace execution-based confirmation.

10.6 State identity, storage cost, and protocol structure

Precise storage identity is a recurring requirement in smart-contract analysis. Symbolic executors and bounded model checkers represent keyed storage with array or slot expressions [14, 17, 29]; MV-SCAN adopts the same principle at a lighter abstraction level by retaining complete nested mapping-key paths and call-context substitutions. Unlike a solver, it records syntactic matches and constrained runtime equalities rather than proving arbitrary key equivalence.

Gas-oriented research explains why developers care about storage layout, caching, repeated accesses, and avoiding unnecessary writes [18]. The EVM gas schedule explicitly prices cold storage access and state changes [10, 34]. This economic context helps explain why protocols maintain checkpoints and derived views, but MV-SI is not defined by gas optimization. The same failure can arise from modular architecture, external mirrors, migration state, or duplicated business rules. For this reason, the present paper treats redundancy as one important source of latent relations rather than as a mandatory or exhaustive root cause.

10.7 Positioning

The nearest lines of work each provide one part of the problem: multi-variable correlation mining identifies related state; ISU studies characterize update failures; same-state detectors reason about ordering and consequential use; static frameworks recover program facts; and symbolic or dynamic tools test path feasibility. MV-SCAN combines these ideas around a different report unit: an annotation-free relation hypothesis, a writer that may update a proper subset, and a sink-influencing read of an omitted member. Its evaluation is correspondingly internal and semantic. We measure whether those candidates reconstruct historical MV-SI and how often sampled structural buckets satisfy C1–C4, rather than treating warnings from adjacent objectives as interchangeable baseline outputs.

11 Conclusion

Multi-variable state inconsistency arises when separately stored protocol state participates in one semantic relation, a feasible transition commits an update to only part of that relation, and omitted state affects behavior before reconciliation. The important boundary is not the local partial write alone. Cached, checkpointed, and lazy state can diverge safely; the defect is the consumption of an invalid joint state.

MV-SCAN operationalizes this boundary as an annotation-free static may-analysis. It infers relation hypotheses from control-sink co-influence and non-mutating return summaries, preserves mapping-key and execution context, requires a nonempty proper-subset writer, tracks omitted members, and retains only persistent reads that may reach control, storage-write, or external-effect sinks. Its writer-centered output records the provenance and constraints needed for review without claiming to prove the relation, path, reconciliation behavior, or exploitability.

The rebuilt evaluation gives a measured account of both capability and limitation. Reviewers classify 64 of 116 published ISU findings as MV-SI; 62 build, and MV-SCAN strictly recovers 13 (21.0%). In the complete configuration, 82 of 400 sampled Web3Bugs structural buckets are MV-SI (20.5% precision). Yet 97.6% of all 1,270 audited relations are semantically supported. The dominant errors occur later: no desynchronizing persistent transition or reconciliation before consumption accounts for 83.9% of nonbugs. Most successful project runs complete in seconds at the median, with a material heavy tail in the ISU corpus.

Annotation-free discovery of latent multi-variable relations is feasible, but relation discovery is not the principal remaining bottleneck. Progress now depends on transaction-level partial-transition, reconciliation, and joint-feasibility reasoning that preserves the state identities and behavioral consumers already exposed by MV-SCAN.

Acknowledgments

References

- [1] 2024. Demystifying Invariant Effectiveness for Securing Smart Contracts. In *Proceedings of the ACM on Software Engineering, FSE*. doi:10.1145/3660786
- [2] 2024. ScType: Towards Finding Accounting Errors in Smart Contracts. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. doi:10.1145/3597503.3639128
- [3] 2024. SmartInv: Multimodal Learning for Smart Contract Invariant Inference. In *2024 IEEE Symposium on Security and Privacy*. doi:10.1109/SP54263.2024.00126
- [4] 2025. Automated Invariant Generation for Solidity Smart Contracts. *IEEE Transactions on Dependable and Secure Computing* (2025). doi:10.1109/TDSC.2025.3592705
- [5] 2025. PropertyGPT: LLM-Driven Formal Verification of Smart Contracts through Retrieval-Augmented Property Generation. In *Network and Distributed System Security Symposium*. doi:10.14722/ndss.2025.241357
- [6] 2025. SmartOracle: Generating Fine-Grained Smart Contract Oracles from Historical Transactions. *IEEE Transactions on Software Engineering* (2025). doi:10.1109/TSE.2025.3534567
- [7] Priyanka Bose, Dipanjan Das, Yanju Chen, Yu Feng, Christopher Kruegel, and Giovanni Vigna. 2022. SAILFISH: Vetting Smart Contract State-Inconsistency Bugs in Seconds. In *2022 IEEE Symposium on Security and Privacy (SP)*. doi:10.1109/SP46214.2022.9833721
- [8] Lexi Brent, Neville Grech, Sifis Lagouvardos, Bernhard Scholz, and Yannis Smaragdakis. 2020. Ethainter: a smart contract security analyzer for composite vulnerabilities. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. doi:10.1145/3385412.3385990
- [9] Lexi Brent, Anton Jurisevic, Michael Kong, Eric Liu, Francois Gauthier, Vincent Gramoli, Ralph Holz, and Bernhard Scholz. 2018. Vandal: A Scalable Security Analysis Framework for Smart Contracts. arXiv:1809.03981 <https://arxiv.org/abs/1809.03981>
- [10] Vitalik Buterin and Martin Swende. 2020. EIP-2929: Gas Cost Increases for State Access Opcodes. Ethereum Improvement Proposal 2929. <https://eips.ethereum.org/EIPS/eip-2929>
- [11] Ethan Cecchetti, Siqui Yao, Haobin Ni, and Andrew C. Myers. 2021. Compositional Security for Reentrant Applications. In *IEEE Symposium on Security and Privacy*. <https://www.cs.cornell.edu/andru/papers/oakland21/>
- [12] Jaeseung Choi, Doyeon Kim, Soomin Kim, Gustavo Grieco, Alex Groce, and Sang Kil Cha. 2021. SMARTIAN: Enhancing Smart Contract Fuzzing with Static and Dynamic Data-Flow Analyses. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. doi:10.1109/ASE51524.2021.9678888
- [13] Code4rena. 2022. Kuiper: Findings & Analysis Report. <https://code4rena.com/reports/2021-12-defiprotocol> Finding H-01: Wrong fee calculation after totalSupply was 0.
- [14] ConsenSys. [n. d.]. Mythril: Security Analysis Tool for EVM Bytecode. Software repository. <https://github.com/ConsenSys/mythril>
- [15] Josselin Feist, Gustavo Grieco, and Alex Groce. 2019. Slither: A Static Analysis Framework for Smart Contracts. In *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. doi:10.1109/WETSEB.2019.00008
- [16] Yu Feng, Emina Torlak, and Rastislav Bodik. 2020. Summary-based symbolic evaluation for smart contracts. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. doi:10.1145/3324884.3416646
- [17] Joel Frank, Cornelius Aschermann, and Thorsten Holz. 2020. ETHBMC: A Bounded Model Checker for Smart Contracts. In *29th USENIX Security Symposium*. 2757–2774. <https://www.usenix.org/conference/usenixsecurity20/presentation/frank>
- [18] Mengting He, Shihao Xia, Boqin Qin, Nobuko Yoshida, Tingting Yu, Yiyang Zhang, and Linhai Song. 2025. How to Save My Gas Fees: Understanding and Detecting Real-World Gas Issues in Solidity Programs. *IEEE Transactions on Software Engineering* 51, 9 (2025), 2617–2633. <https://mrg.cs.ox.ac.uk/publications/how-to-save-my-gas-fees-understanding-and-detecting-real-world-gas-issues-in-solidity-programs/main.pdf>
- [19] Bo Jiang, Ye Liu, and W. K. Chan. 2018. ContractFuzzer: fuzzing smart contracts for vulnerability detection. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. doi:10.1145/3238147.3238177
- [20] Sukrit Kalra, Seep Goel, Mohan Dhawan, and Subodh Sharma. 2018. ZEUS: Analyzing Safety of Smart Contracts. In *Network and Distributed System Security Symposium*. https://www.ndss-symposium.org/wp-content/uploads/2018/02/ndss2018_09-1_Kalra_paper.pdf
- [21] Aashish Kolluri, Ivica Nikolic, Ilya Sergey, Aquinas Hobor, and Prateek Saxena. 2019. Exploiting the laws of order in smart contracts. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. doi:10.1145/3293882.3330560
- [22] Johannes Krupp and Christian Rossow. 2018. teEther: Gnawing at Ethereum to Automatically Exploit Smart Contracts. In *27th USENIX Security Symposium*. <https://www.usenix.org/conference/usenixsecurity18/presentation/krupp>
- [23] Lantian Li, Yuyu Chen, Jingwen Wu, Yue Pan, and Zhongxing Yu. 2026. Understanding Inconsistent State Update Vulnerabilities in Smart Contracts. *ACM Transactions on Software Engineering and Methodology* (2026). doi:10.1145/3798052
- [24] Zeqin Liao, Zibin Zheng, Xiao Chen, and Yuhong Nan. 2022. SmartDagger: a bytecode-based static analysis approach for detecting cross-contract vulnerability. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. doi:10.1145/3533767.3534222
- [25] Yinxi Liu, Wei Meng, and Yinqian Zhang. 2025. Detecting Smart Contract State-Inconsistency Bugs via Flow Divergence and Multiplex Symbolic Execution. *Proceedings of the ACM on Software Engineering* (2025). doi:10.1145/3715712

- [26] Shan Lu, Soyeon Park, Chongfeng Hu, Xiao Ma, Weihang Jiang, Zhenmin Li, Raluca A. Popa, and Yuanyuan Zhou. 2007. MUVI. In *Proceedings of twenty-first ACM SIGOPS symposium on Operating systems principles*. doi:10.1145/1294261.1294272
- [27] Loi Luu, Duc-Hiep Chu, Hrishikesh Olickel, Prateek Saxena, and Aquinas Hobor. 2016. Making Smart Contracts Smarter. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. doi:10.1145/2976749.2978309
- [28] Chenyang Ma, Wei Song, and Jeff Huang. 2023. TransRacer: Function Dependence-Guided Transaction Race Detection for Smart Contracts. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. doi:10.1145/3611643.3616281
- [29] Mark Mossberg, Felipe Manzano, Eric Hennenfent, Alex Groce, Gustavo Grieco, Josselin Feist, Trent Brunson, and Artem Dinaburg. 2019. Manticore: A User-Friendly Symbolic Execution Framework for Binaries and Smart Contracts. In *34th IEEE/ACM International Conference on Automated Software Engineering*. <https://arxiv.org/abs/1907.03890>
- [30] Sundas Munir and Christoph Reichenbach. 2023. TODLER: A Transaction Ordering Dependency anaLyzER - for Ethereum Smart Contracts. In *2023 IEEE/ACM 6th International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. doi:10.1109/WETSEB59161.2023.00007
- [31] Tai D. Nguyen, Long H. Pham, Jun Sun, Yun Lin, and Quang Tran Minh. 2020. sFuzz. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. doi:10.1145/3377811.3380334
- [32] Michael Rodler, Wenting Li, Ghassan O. Karame, and Lucas Davi. 2019. Sereum: Protecting Existing Smart Contracts Against Re-Entrancy Attacks. In *Proceedings 2019 Network and Distributed System Security Symposium*. doi:10.14722/ndss.2019.23413
- [33] Sunbeom So, Seongjoon Hong, and Hakjoo Oh. 2021. SmarTest: Effectively Hunting Vulnerable Transaction Sequences in Smart Contracts through Language Model-Guided Symbolic Execution. In *30th USENIX Security Symposium*. <https://www.usenix.org/conference/usenixsecurity21/presentation/so>
- [34] Wei Tang. 2019. EIP-2200: Structured Definitions for Net Gas Metering. Ethereum Improvement Proposal 2200. <https://eips.ethereum.org/EIPS/eip-2200>
- [35] Sergei Tikhomirov, Ekaterina Voskresenskaya, Ivan Ivanitskiy, Ramil Takhaviev, Evgeny Marchenko, and Yaroslav Alexandrov. 2018. SmartCheck. In *Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain*. doi:10.1145/3194113.3194115
- [36] Trail of Bits. [n. d.]. Echidna: Ethereum Smart Contract Fuzzer. Software repository. <https://github.com/crytic/echidna>
- [37] Petar Tsankov, Andrei Dan, Dana Drachler Cohen, Arthur Gervais, Florian Bueznli, and Martin Vechev. 2018. Securify: Practical Security Analysis of Smart Contracts. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. <https://files.sri.inf.ethz.ch/website/papers/ccs18-securify.pdf>
- [38] Shuai Wang, Chengyu Zhang, and Zhendong Su. 2019. Detecting nondeterministic payment bugs in Ethereum smart contracts. *Proceedings of the ACM on Programming Languages* (2019). doi:10.1145/3360615
- [39] Valentin Wüstholz and Maria Christakis. 2020. Harvey: a greybox fuzzer for smart contracts. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. doi:10.1145/3368089.3417064
- [40] Valentin Wüstholz and Maria Christakis. 2020. Targeted greybox fuzzing with static lookahead analysis. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. doi:10.1145/3377811.3380388
- [41] Yinxiang Xue, Mingliang Ma, Yun Lin, Yulei Sui, Jiaming Ye, and Tianyong Peng. 2020. Cross-contract static analysis for detecting practical reentrancy vulnerabilities in smart contracts. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. doi:10.1145/3324884.3416553
- [42] Wuqi Zhang, Lili Wei, Shing-Chi Cheung, Yepang Liu, Shuqing Li, Lu Liu, and Michael R. Lyu. 2023. Combatting Front-Running in Smart Contracts: Attack Mining, Benchmark Construction and Vulnerability Detector Evaluation. *IEEE Transactions on Software Engineering* (2023). doi:10.1109/TSE.2023.3270117
- [43] Zhuo Zhang, Brian Zhang, Wen Xu, and Zhiqiang Lin. 2023. Demystifying Exploitable Bugs in Smart Contracts. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. doi:10.1109/ICSE48619.2023.00061

A Artifact and Reproducibility Information

[VU TODO: Add the archival link and exact reproduction commands once packaged.]